

Guía de Estudio y Banco de Preguntas Desarrolladas: Primer Parcial de Ingeniería y Calidad de Software (Unidades 1, 2 y 3)

1. UNIDAD 1: INGENIERÍA DE SOFTWARE EN CONTEXTO

Pregunta 1: Definición de Ingeniería de Software y Diferencias con la Manufactura Tradicional

Consigna Formal:

Defina el término *Ingeniería de Software* a partir de la formulación oficial del IEEE formalizada en el SWEBOK y la perspectiva conceptual de David Parnas. Asimismo, analice y justifique las razones técnico-conceptuales por las cuales el desarrollo de software no puede compararse o equipararse directamente con los procesos de manufactura industrial tradicional.

Respuesta Desarrollada y Fundamentación Conceptual:

La **Ingeniería de Software (ISW)** se define formalmente, según el *Software Engineering Body of Knowledge (SWEBOK)* de la IEEE, como: "*La aplicación de un enfoque sistemático, disciplinado y cuantificable al desarrollo, operación y mantenimiento del software; es decir, la aplicación de la ingeniería al software*". Por su parte, David Parnas ofrece una visión sintética y centrada en la realidad operativa de la industria al definirla como: "*Multi-person construction of multi-version software*" (Construcción por múltiples personas de software multiversión). A diferencia de los productos manufacturados en industrias tradicionales (como la automotriz o la metalmecánica), el software posee propiedades intrínsecas que impiden tratar su construcción como una línea de ensamblaje masiva. Tal como se evalúa en las instancias teóricas de la cátedra de la UTN-FRC, el software es un producto puramente intelectual cuyas diferencias conceptuales de fondo respecto a la manufactura se detallan a continuación:

Factor Diferenciador	Fundamentación Conceptual
Desecho y Desgaste (Wear-out)	El software no se gasta con el uso ni sufre degradación física por rozamiento, fricción o factores ambientales. Las fallas que surgen con el tiempo se deben a cambios en el entorno operativo o a errores latentes de diseño e implementación, no al deterioro del material.
Producción en Masa y Reproducibilidad	No existe la producción masiva en software. Una vez que la unidad lógica (código) es desarrollada, su duplicación/copia es de costo prácticamente nulo. Casi ningún producto de software es idéntico a otro, ya que la fase costosa e ingenieril es el diseño e intelecto conceptual, no la fabricación.
Leyes de la Física	El software es un elemento intangible y abstracto. No está gobernado por las leyes físicas de la naturaleza (gravedad, resistencia de materiales, termodinámica), lo que imposibilita la aplicación directa de las fórmulas y modelos de la ingeniería tradicional.
Factor Humano e Intelecto	La creación de software es una empresa puramente intelectual y humana. Depende intrínsecamente de las personalidades, la capacidad analítica y la dinámica de los miembros del equipo. Un proceso de desarrollo que resulta altamente exitoso en un equipo puede fracasar al aplicarse a otro grupo con distintas dinámicas.
Naturaleza de las Fallas	No todas las fallas en el software corresponden a errores puntuales de programación (<i>bugs</i>). Muchas fallas derivan de inconsistencias en la especificación, fallas de comunicación,

requerimientos ambiguos o comportamientos emergentes no previstos en sistemas complejos. |

Pregunta 2: Antecedentes Históricos y la Crisis del Software de 1968

Consigna Formal:

Sintetice el contexto histórico en el que surge el término "Ingeniería de Software" durante la Conferencia de la NATO de 1968. Explique los problemas sistemáticos asociados a la llamada "Crisis del Software" y enumere las causas estadísticas principales que fundamentan los fracasos en los proyectos de desarrollo de software.

Respuesta Desarrollada y Fundamentación Conceptual:

El término **Ingeniería de Software** fue acuñado formalmente en octubre de **1968** durante una histórica conferencia patrocinada por el Comité de Ciencia de la **NATO** (Organización del Tratado del Atlántico Norte) celebrada en Garmisch, Alemania. El encuentro se convocó de emergencia para abordar la preocupación generalizada sobre la "Crisis del Software", un fenómeno caracterizado por la incapacidad sistemática de la industria de entregar sistemas funcionales dentro de los parámetros estipulados de tiempo, presupuesto y calidad. Los problemas recurrentes manifestados en la práctica del desarrollo de software que dieron origen a esta crisis se sintetizan en los siguientes puntos críticos:

- **Mala satisfacción de necesidades:** La versión final del producto entregado habitualmente no satisface las necesidades reales ni las expectativas del cliente o usuario final.
- **Inextensibilidad e inflexibilidad:** El software no es fácil de extender y/o adaptar. Agregar nueva funcionalidad en versiones posteriores se convierte en una tarea sumamente compleja o técnicamente inviable.
- **Deficiencia documental:** Existencia de mala documentación o ausencia total de la misma, imposibilitando el mantenimiento y la evolución del sistema.
- **Mala calidad técnica:** Productos entregados con alta densidad de defectos, inestabilidad operativa y baja confiabilidad.
- **Desvíos presupuestarios y temporales:** Proyectos que insumen significativamente más tiempo y costos que los presupuestados originariamente. Las investigaciones estadísticas sobre las causas por las cuales los proyectos fracasan demuestran que las deficiencias no se ubican primordialmente en la fase de codificación sintáctica, sino en la gestión e ingeniería inicial de requerimientos. Las proporciones de las causas principales de falla son:

1. **Requerimientos incompletos:** 13.1 %
 2. **Falta de involucramiento del usuario:** 12.4 %
 3. **Falta de recursos:** 10.6 %
 4. **Expectativas poco realistas:** 9.3 %
 5. **Falta de apoyo de la Gerencia:** 8.7 %
 6. **Requerimientos cambiantes:** 8.1 %
- Estas evidencias sustentan la máxima fundamental sostenida por la cátedra: *"Saber programar NO es suficiente"* .

Pregunta 3: El Ensayo "No Silver Bullet" de Fred Brooks

Consigna Formal:

Analice el ensayo "No Silver Bullet: Essence and Accidents of Software Engineering" de Frederick Brooks (1987). Desarrolle la distinción conceptual entre Dificultades Esenciales y Dificultades Accidentales, y explique la tesis principal de Brooks sobre la evolución de la productividad en la Ingeniería de Software.

Respuesta Desarrollada y Fundamentación Conceptual:

En su ensayo de 1987, Frederick Brooks sostiene la tesis central de que **no existe ninguna innovación técnica o de gestión por sí sola (ninguna "bala de plata") que prometa mejoras de un orden de magnitud (10x) en productividad, fiabilidad o simplicidad en el desarrollo de software dentro de una década**. Para fundamentar su postura, Brooks categoriza los problemas de la construcción de software en dos dominios claramente delimitados:

- **Dificultades Esenciales:** Son aquellas inherentes a la naturaleza misma del software y que constituyen su esencia conceptual abstracta:
- **Complejidad:** Ningún par de partes del software es idéntico. La complejidad no escala de forma lineal con el tamaño del sistema, sino de forma exponencial.
- **Conformidad:** El software debe amoldarse y ajustarse a interfaces humanas, sistemas legados, reglamentaciones e instituciones arbitrarias que no fueron diseñadas de manera racional o unificada.
- **Adaptabilidad (*Changeability* / Inconstancia):** El software está sometido a una presión constante de cambio debido a que se percibe como un elemento altamente flexible en comparación con el hardware.
- **Invisibilidad:** El software es inmaterial e invisible. No posee una representación geométrica espacial directa, lo que dificulta enormemente su visualización gráfica y modelado integral.
- **Dificultades Accidentales:** Son aquellas trabas asociadas a la tecnología utilizada para construir el software en una época determinada, tales como las limitaciones de los lenguajes de programación, la lentitud de los procesos de compilación, la torpeza de los entornos de desarrollo (IDE) o las restricciones físicas del hardware. Brooks concluye que los grandes avances históricos de la disciplina lograron eliminar dificultades accidentales (por ejemplo, el paso del lenguaje ensamblador a lenguajes de alto nivel), pero las dificultades esenciales continúan presentes. Por ende, ninguna herramienta o metodología mágica podrá resolver drásticamente los problemas que derivan de la esencia conceptual abstracta del software.

Pregunta 4: Disciplinas Técnicas, de Gestión y de Soporte en ISW

Consigna Formal:

Clasifique y describa las disciplinas de la Ingeniería de Software agrupándolas según su función de soporte, gestión o ejecución técnica a lo largo del ciclo de vida del producto informático.

Respuesta Desarrollada y Fundamentación Conceptual:

La Ingeniería de Software integra un conjunto de disciplinas interconectadas organizadas para garantizar la calidad del producto y la viabilidad de la ejecución del proyecto a lo largo de su ciclo de vida. Se estructuran en tres bloques principales:

- **Disciplinas Técnicas:** Enfocadas en el flujo directo de transformación de la necesidad del cliente en un producto de software operativo:
- **Ingeniería de Requerimientos:** Descubrimiento, análisis, especificación y validación de las capacidades y restricciones del sistema.
- **Diseño de Software:** Definición de la arquitectura interna, componentes, interfaces, patrones y estructuras de datos.
- **Construcción / Desarrollo:** Codificación, implementación de la lógica de negocio y pruebas unitarias de los módulos.
- **Despliegue (*Deployment*):** Estrategias, empaquetado e infraestructura para poner el producto en entornos de producción operativos.
- **Validación y Verificación (Pruebas / *Testing*):** Evaluación del software para asegurar que cumple las especificaciones y responde a las necesidades del cliente.
- **Disciplinas de Gestión:** Enfocadas en la administración de los recursos, compromisos, estimaciones y agendas:
- **Gestión de Proyectos:** Planificación, seguimiento del progreso, gestión de riesgos y dirección general.
- **Estimaciones y Métricas:** Cuantificación del tamaño, esfuerzo, tiempos, costos y medición de atributos de proceso y producto.
- **Planificación de Desarrollo y Mantenimiento:** Estructuración de la evolución y soporte continuo del sistema a lo largo de su vida útil.
- **Disciplinas de Soporte y Transversales:** Enfocadas en garantizar la calidad del proceso, la integridad de los artefactos y la gobernanza:
- **Gestión de Configuración de Software (SCM):** Identificación, control de versiones, trazabilidad y auditoría de los cambios en los artefactos.
- **Aseguramiento de la Calidad del Software (SQA):** Auditoría y revisión sistemática del cumplimiento de procesos e indicadores de calidad.
- **Auditorías de Software:** Revisiones formales independientes sobre procesos, proyectos y calidad de código o entregables.

Pregunta 5: Ciclos de Vida y Modelos de Proceso Tradicionales vs. Ágiles

Consigna Formal:

Defina el concepto de Ciclo de Vida del Software y analice su impacto en la administración de proyectos. Compare las características de los enfoques tradicionales (predictivos/definidos) respecto a los enfoques ágiles (adaptativos/empíricos) mediante una tabla comparativa.

Respuesta Desarrollada y Fundamentación Conceptual:

Un **Ciclo de Vida del Software (o Modelo de Proceso)** se define conceptualmente como la representación abstracta de un proceso de desarrollo. Específicamente, determina las fases del proyecto, su orden de ejecución y las actividades asociadas a cada una, desde la concepción inicial hasta el retiro del sistema. Como se reafirma en las evaluaciones teóricas

de la UTN-FRC, los ciclos de vida no definen técnicas de programación, sino que influyen de manera determinante en la administración del proyecto.

- **Enfoques Tradicionales (Predictivos / Definidos):** Asumen que los requerimientos pueden ser comprendidos y congelados de entrada. Avanzan en fases secuenciales rígidas con alta carga de documentación formal.
- **Enfoques Ágiles (Adaptativos / Empíricos):** Aceptan la incertidumbre inherente del software. Entregan valor funcional en iteraciones cortas e incrementales, adaptando el plan continuamente mediante la inspección del producto. | Eje de Comparación | Ciclo de Vida Tradicional (Predictivo / Definido) | Ciclo de Vida Ágil (Iterativo / Incremental) || ----- | ----- | ----- || **Ventajas** | Estructura predecible; facilita contratos a precio fijo; requerimientos y alcance firmes desde el inicio; documentación formal exhaustiva. | Entrega rápida e incremental de valor funcional; capacidad de adaptación y aprendizaje continuo; visibilidad constante; validación temprana con el usuario. || **Desventajas** | Alta inflexibilidad ante cambios; los errores de análisis se detectan en fases tardías; las pruebas e integración se posponen al final del proyecto. | Requiere alta disponibilidad y compromiso del cliente; dificultad para predecir tiempo y costo total iniciales; riesgo de deuda técnica si no se refactoriza. || **Tolerancia al Cambio** | Muy baja. Los cambios en etapas avanzadas son sumamente costosos y requieren procedimientos burocráticos formales de control de cambios. | Muy alta. El cambio se acepta como un elemento natural del desarrollo y se aprovecha para maximizar el valor de negocio en cada nueva iteración. |

Pregunta 6: Criterios de Selección de Ciclos de Vida

Consigna Formal:

Detalle los criterios formales que debe considerar un líder de proyecto para seleccionar el ciclo de vida o modelo de proceso más adecuado para un desarrollo de software concreto.

Respuesta Desarrollada y Fundamentación Conceptual:

La selección del modelo de proceso no debe responder a modas tecnológicas, sino a un análisis riguroso del contexto del proyecto. Los criterios fundamentales establecidos por la cátedra son:

1. **Claridad y Estabilidad de Requerimientos:** Si los requerimientos están completamente definidos, son estables y de bajo riesgo de modificación, los modelos tradicionales (Cascada) son viables. Si existe alta volatilidad, ambigüedad o necesidad de exploración, se imponen modelos ágiles (Scrum).
2. **Complejidad Técnica y Novedad:** Proyectos que involucran tecnologías emergentes, arquitecturas no probadas o integración con sistemas de terceros de comportamiento desconocido requieren ciclos iterativos que permitan realizar prototipos y mitigar riesgos técnicos.
3. **Nivel de Riesgo del Proyecto:** Proyectos con altos factores de incertidumbre operativa o de mercado demandan ciclos de vida incrementales para validar hipótesis de manera temprana.
4. **Críticidad del Producto:** En sistemas críticos donde una falla puede provocar pérdidas humanas o desastres financieros severos (ej. aviación, salud, sistemas bancarios centrales), los procesos exigen controles rigurosos, fases secuenciales formalmente auditadas e intencionalidad documental.

5. **Involucramiento y Disponibilidad del Cliente/Usuario:** Los enfoques ágiles exigen la participación activa y continua del cliente (o Product Owner) en cada iteración. Si el cliente solo puede intervenir al inicio para definir requerimientos y al final para aceptar el producto, un enfoque predictivo resulta necesario.
6. **Experiencia Previa del Equipo de Desarrollo:** El nivel de madurez técnica del equipo, su dominio de la tecnología y su capacidad de autoorganización condicionan la elección. Un equipo sin experiencia en marcos ágiles fracasará si no cuenta con el acompañamiento adecuado.

Pregunta 7: Procesos de Desarrollo Empíricos vs. Definidos

Consigna Formal:

Establezca las diferencias teóricas y operativas entre un proceso de desarrollo *definido* y un proceso de desarrollo *empírico*, fundamentando los tres pilares en los que se sostiene el control de procesos empírico.

Respuesta Desarrollada y Fundamentación Conceptual:

La distinción entre control de procesos definido y empírico radica en la naturaleza del entorno en el que se ejecutan:

- **Control de Procesos Definido:** Es aplicable a entornos totalmente repetibles, predecibles y comprendidos de extremo a extremo. Asume que, dada una misma entrada (*input*), la ejecución del proceso predeterminado generará exactamente la misma salida (*output*). Requiere una planificación inicial cerrada y detallada.
- **Control de Procesos Empírico:** Es aplicable a entornos complejos, no lineales y con alta incertidumbre (como la creación de software). Se basa en la premisa de que no se puede prever de antemano todo el comportamiento del sistema. Por ende, las decisiones se toman basadas en la observación real, la experiencia de campo y la experimentación. El control de procesos empírico se sostiene sobre **Tres Pilares Fundamentales** :

1. **Transparencia:** Todos los aspectos del proceso y del producto deben ser visibles y comprensibles para aquellos que gestionan y ejecutan el trabajo. Exige el uso de un lenguaje común y definiciones compartidas (como la *Definition of Done*).
2. **Inspección:** Los artefactos del proceso y el avance hacia las metas deben ser inspeccionados con frecuencia para detectar desviaciones indeseadas o problemas de calidad de forma oportuna.
3. **Adaptación:** Si la inspección revela que uno o más aspectos del proceso o del producto se han desviado fuera de los límites aceptables, el proceso, las herramientas o los materiales deben ajustarse inmediatamente para minimizar desviaciones futuras.

Pregunta 8: Vínculo Proceso-Proyecto-Producto y la Triple Restricción

Consigna Formal:

Explique el vínculo interdependiente Proceso-Proyecto-Producto dentro del marco de la gestión de software y describa el impacto que ejerce el dilema de la Triple Restricción en la calidad del sistema.

Respuesta Desarrollada y Fundamentación Conceptual:

En la gestión de desarrollo de software existe un triángulo interdependiente:

- **Proceso:** Es el conjunto estructurado de actividades, métodos, prácticas y herramientas que guían la ejecución de las tareas técnicas.
 - **Proyecto:** Es la entidad organizativa temporal que aplica el proceso para administrar eficientemente los recursos humanos, el presupuesto, el tiempo y los riesgos.
 - **Producto:** Es el resultado final (código ejecutable, arquitectura, documentación, módulos) obtenido mediante la ejecución del proyecto a través del proceso. Esta dinámica está gobernada por la **Triple Restricción Clásica** (Alcance, Tiempo y Costo/Recursos):
 - **Alcance:** Funcionalidades y características que el software debe incluir.
 - **Tiempo:** Ventana temporal o fecha límite estipulada para la entrega.
 - **Costo / Recursos:** Presupuesto asignado y capacidad del equipo humano disponible.
- Impacto en la Calidad y Contratos:** La **Calidad** se posiciona en el centro de la Triple Restricción. En los contratos tradicionales de precio y alcance fijo, al intentar congelar simultáneamente el Alcance, el Tiempo y el Costo, cualquier imprevisto técnico fuerza la degradación implícita de la Calidad (recorte de pruebas, código no refactorizado, ausencia de documentación). En contraste, los enfoques ágiles fijan el Tiempo y el Costo/Recursos por iteración, manteniendo el **Alcance flexible**, lo que permite resguardar la Calidad del Producto como un atributo no negociable.

2. UNIDAD 2: GESTIÓN LEAN-ÁGIL DE PRODUCTOS DE SOFTWARE

Pregunta 9: Filosofía Lean y Gestión de Productos de Software

Consigna Formal:

Describa los principios fundamentales de la Filosofía Lean aplicados a la gestión de productos de software. Explique detalladamente el concepto de "desperdicio" (*waste*) y evalúe los riesgos de liberar funcionalidades completas sin realizar la validación previa de hipótesis de valor.

Respuesta Desarrollada y Fundamentación Conceptual:

La **Filosofía Lean**, adaptada al software por Mary y Tom Poppendieck, se centra en eliminar el desperdicio, optimizar la entrega de valor y acelerar el aprendizaje validado. En software, se define como **desperdicio (*waste*)** a todo aquello que consume tiempo, esfuerzo o recursos pero **no agrega valor directo percibido por el usuario final** (ej. burocracia documental, código no utilizado, esperas por aprobaciones). **Riesgos de Liberar Funcionalidades sin Validar Hipótesis de Valor:** De acuerdo con las evaluaciones teóricas oficiales de la UTN-FRC (Tema A, Pregunta 5), cuando un equipo desarrolla e implementa una característica (*feature*) completa sin validar previamente la hipótesis de valor con los usuarios, aumentan drásticamente los siguientes riesgos:

- **Generación masiva de desperdicio:** Inversión de esfuerzo ingenieril en funcionalidades que no aportan valor real.
- **Time-to-Market excesivo:** Retraso sustancial en el lanzamiento del producto intentando perfeccionar características cuya utilidad no ha sido comprobada.

- **Construir algo que nadie quiere:** Es el riesgo más severo en la gestión de productos; entregar un software técnicamente impecable pero comercial y funcionalmente irrelevante para el mercado.
- **Sobre-ingeniería (Over-engineering):** Diseñar soluciones técnicas hipercomplejas y costosas para problemas no verificados.

Pregunta 10: Estrategias de Liberación: MVP, MMF y MLP

Consigna Formal:

Defina y diferencie los conceptos de Producto Mínimo Viable (MVP), Característica Mínima Comercializable (MMF) y Producto Mínimo Querible (MLP). Elabore una tabla comparativa que contraste sus objetivos, foco de entrega y público objetivo.

Respuesta Desarrollada y Fundamentación Conceptual:

En la gestión moderna de productos se distinguen tres estrategias de liberación según el objetivo perseguido:

- **MVP (Minimum Viable Product):** Es una versión preliminar diseñada para **maximizar el aprendizaje validado** sobre los clientes con el **mínimo esfuerzo inicial**. No es un producto comercial completo, sino un experimento orientado a probar una hipótesis (Tema A, Pregunta 4).
 - **MMF (Minimum Marketable Feature):** Es un conjunto reducido, funcionalmente completo y coherente de capacidades del software que entrega **valor comercial concreto e incremental** a los usuarios. Representa algo desplegable y operativamente utilizable en el mercado (Tema B, Pregunta 4).
 - **MLP (Minimum Lovable Product):** Es un enfoque que prioriza generar una **conexión y deleite emocional inicial** en los usuarios a través de una experiencia de usuario (UX/UI) cuidada desde el primer lanzamiento.
- | Concepto | Objetivo Principal | Foco de Entrega | Audiencia Destino |
|--|--|-----------------|-------------------|
| MVP (Producto Mínimo Viable) | Maximizar el aprendizaje validado sobre una hipótesis con el mínimo esfuerzo inicial. Experimento básico, prototipo o funcionalidad mínima de verificación. <i>Early adopters</i> (adoptantes tempranos) dispuestos a probar soluciones iniciales. | | |
| MMF (Característica Mínima Comercializable) | Aportar valor comercial concreto e incremental de forma utilizable. Funcionalidad madura, integrable, operable y desplegable a producción. Clientes activos y mercado general que buscan resolver una necesidad puntual. | | |
| MLP (Producto Mínimo Querible) | Generar deleite emocional, adopción acelerada y fidelidad desde el inicio. Experiencia de usuario (UX) cuidada, usabilidad superior y diseño atractivo. Usuarios finales exigentes en mercados altamente competitivos. | | |

Pregunta 11: Requerimientos en Ambientes Ágiles y User Stories (US)

Consigna Formal:

Analice la naturaleza de los requerimientos en entornos ágiles mediante el concepto de Historia de Usuario (User Story - US). Establezca las diferencias fundamentales entre una User Story y una ERS (Especificación de Requerimientos de Software) tradicional basada en estándares como el IEEE 830.

Respuesta Desarrollada y Fundamentación Conceptual:

En entornos Lean-Ágiles, los requerimientos no se conciben como contratos estáticos e inmutables redactados al inicio del proyecto. Una **Historia de Usuario (User Story)** es una representación simplificada de alto nivel de una necesidad del usuario que expresa un valor de negocio explícito (Tema A, Pregunta 2). Sus diferencias sustanciales frente a una **ERS (Especificación de Requerimientos de Software)** tradicional son:

- **Descripción de la Necesidad vs. Solución Técnica:**
 - La **ERS** pretende definir en un documento exhaustivo la solución técnica detallada antes de programar.
 - La **User Story** se limita a describir la **necesidad del usuario** y el valor de negocio buscado, postergando la definición del detalle técnico para el momento de la conversación e implementación.
- **Facilitadora de Conversación vs. Reemplazo de Conversación:**
 - La **ERS** actúa como un contrato escrito que intenta "reemplazar" el diálogo directo entre analistas y desarrolladores.
 - La **User Story** funciona intencionalmente como una invitación al diálogo, **facilitando la conversación** continua entre el *Product Owner*, desarrolladores y *testers*.
- **Flexibilidad y Adaptabilidad:**
 - Las Historias de Usuario son negociables y livianas; los documentos ERS generan rigidez y burocracia ante los cambios de alcance.

Pregunta 12: La Estructura de las 3Cs en Historias de Usuario

Consigna Formal:

Desarrolle el modelo conceptual de las 3Cs formulado por Ron Jeffries para estructurar las dimensiones operativas de una Historia de Usuario (Card, Conversation, Confirmation).

Respuesta Desarrollada y Fundamentación Conceptual:

Ron Jeffries definió el modelo de las **3Cs** para dejar en claro que una Historia de Usuario no es únicamente una frase escrita en una tarjeta, sino un proceso compuesto por tres dimensiones integradas:

- **Card (Tarjeta):**
 - Es el soporte físico o digital que sintetiza el concepto de la historia. Funciona como un recordatorio visible de la necesidad del usuario y el beneficio de negocio perseguido.
 - Responde a la estructura estándar: "*Como rol de usuario, quiero acción / funcionalidad, para beneficio esperado / valor de negocio*".
- **Conversation (Conversación):**
 - Representa el intercambio de conocimientos verbal e interactivo entre el *Product Owner*, los desarrolladores y los *testers*.
 - Durante la conversación se aclaran las dudas, se debaten las alternativas arquitectónicas, se analizan los riesgos y se refinan los detalles de implementación.
- **Confirmation (Confirmación):**
 - Constituye el conjunto explícito de **Criterios de Aceptación** que determinan las condiciones objetivas que debe satisfacer la historia para ser considerada terminada y aprobada.

- Proveen la frontera de alcance de la historia y sirven de base directa para el diseño de los casos de prueba.

Pregunta 13: Aplicación de la Regla INVEST para User Stories

Consigna Formal:

Desglose analíticamente el acrónimo INVEST para la formulación de Historias de Usuario de alta calidad, explicitando el significado y la importancia de cada uno de sus atributos.

Respuesta Desarrollada y Fundamentación Conceptual:

El acrónimo **INVEST** (propuesto por Bill Wake) establece las características de calidad que debe reunir una Historia de Usuario bien formulada:

- **I - Independent (Independiente):** Las historias deben estar desacopladas entre sí. Las dependencias cruzadas dificultan su priorización y estimación individual en el *Product Backlog*.
- **N - Negotiable (Negociable):** La historia no es un contrato cerrado. Sus detalles específicos se negocian y ajustan mediante la conversación entre el equipo y el *Product Owner*.
- **V - Valuable (Valiosa):** Debe aportar un valor de negocio explícito y comprensible para el usuario o el cliente.
- **E - Estimable (Estimable):** El equipo de desarrollo debe comprender el alcance lo suficiente como para emitir una estimación relativa de su tamaño o complejidad.
- **S - Small / Small enough (Pequeña):** Su tamaño debe ser lo suficientemente reducido como para ser diseñada, codificada, probada e integrada completamente dentro de un único Sprint.
- **T - Testable (Comprobable / Testable):** Debe contar con criterios de aceptación claros que permitan verificar de forma objetiva si la funcionalidad implementada es correcta.

Pregunta 14: Criterios de Aceptación y Segmentación Vertical (Vertical Slicing)

Consigna Formal:

Explique el rol técnico de los Criterios de Aceptación en las Historias de Usuario y desarrolle la técnica de Segmentación Vertical (*Vertical Slicing*) contrapuesta a la segmentación horizontal por capas arquitectónicas.

Respuesta Desarrollada y Fundamentación Conceptual:

Rol de los Criterios de Aceptación: Según se establece en la bibliografía de la cátedra (Tema B, Pregunta 2), los Criterios de Aceptación cumplen las siguientes funciones clave:

- Definir cuándo una historia está completa y lista para su aprobación.
 - Alinear la comprensión funcional entre el *Product Owner*, los *testers* y los desarrolladores.
 - Documentar detalles, restricciones y casos de borde que no quedan explicitados en el enunciado corto de la tarjeta (*Card*).
- Segmentación Vertical (*Vertical Slicing*):** A diferencia del enfoque tradicional que divide el trabajo de forma horizontal por capas de software (ej. construir la Base de Datos en un Sprint, la API en otro y la Interfaz gráfica en un tercero), el enfoque Lean-Ágil exige aplicar **Vertical Slicing**.

- **Definición de Segmentación Vertical:** Consiste en cortar la funcionalidad atravesando **todas las capas de la arquitectura** (Interfaz de Usuario, Lógica de Negocio, Capa de Datos e Integración) en una sola historia delgada de punta a punta (*end-to-end*).
- **Justificación Conceptual:** Una capa horizontal aislada (como una tabla de base de datos) no entrega valor utilizable por sí misma. La segmentación vertical garantiza que al finalizar la historia se entregue un incremento funcional integrable, operable y testeable que genera valor real y permite obtener retroalimentación inmediata del usuario.

Pregunta 15: Estimaciones Ágiles: Story Points vs. Horas Hombre

Consigna Formal:

Defina el concepto de *Story Point* en estimaciones ágiles y explicita las variables que abarca. Discuta críticamente las consecuencias operativas perjudiciales de forzar la conversión directa de Story Points a horas hombre fijas por parte de la gestión de proyectos.

Respuesta Desarrollada y Fundamentación Conceptual:

Un **Story Point (Punto de Historia)** es una unidad de medida relativa y abstracta utilizada por los equipos de desarrollo para estimar el **tamaño global** de una Historia de Usuario (Tema A, Pregunta 3). Los Story Points no representan tiempo directamente; abarcan de forma combinada tres dimensiones:

1. **Complejidad:** Dificultad técnica o algorítmica de la solución.
 2. **Esfuerzo:** Cantidad de trabajo físico o mental a realizar.
 3. **Incertidumbre, Duda y Riesgo:** Grado de desconocimiento técnico o volatilidad de los requerimientos. Los Story Points son estrictamente **específicos de cada equipo**. Por esta razón, no es técnicamente válido utilizarlos para comparar la productividad de equipos distintos.
- Consecuencias de forzar la conversión de Story Points a Horas Hombre:** En las evaluaciones teóricas de la UTN-FRC (Tema B, Pregunta 3) se enfatiza que exigir la conversión directa de Story Points a horas para comprometer fechas contractuales genera problemas graves:
- **Confusión entre tamaño y esfuerzo/tiempo:** Se destruye la abstracción relativa, obligando al equipo a pensar en cronogramas rígidos en lugar de complejidad intrínseca.
 - **Uso indebido de las estimaciones:** Las estimaciones dejan de ser herramientas de planificación empírica y se convierten en mecanismos de control.
 - **Presión indebida sobre el equipo:** Se generan compromisos irreales que provocan sobretrabajo, atajos técnicos y caída de la calidad.
 - **Pérdida del enfoque empírico:** Se abandona la medición histórica de la *Velocidad* real del equipo (puntos completados por Sprint), reemplazándola por proyecciones teóricas infundadas.

Pregunta 16: Dinámica de Estimación: Planning Poker y Cono de Incertidumbre

Consigna Formal:

Explique la mecánica de la técnica de estimación *Planning Poker* y describa la utilidad del gráfico del *Cono de Incertidumbre*. Defina el concepto de *Spike* técnico en el contexto de gestión de dudas e incertidumbre.

Respuesta Desarrollada y Fundamentación Conceptual:

Planning Poker: Es una técnica de estimación consensuada basada en Delphi de banda ancha. Cada desarrollador posee un mazo de cartas con la secuencia de Fibonacci modificada (0, 1, 2, 3, 5, 8, 13, 20, 40, 100).

- **Mecánica:** Se expone la Historia de Usuario y el Product Owner aclara dudas. Cada estimador selecciona una carta en privado para evitar el sesgo de anclaje (*anchoring*). Se muestran las cartas en simultáneo. Si existen discrepancias marcadas (ej. un 2 frente a un 13), los votantes de los valores extremos justifican sus razones técnicas. Se debate y se vuelve a votar hasta alcanzar consenso.
- El Cono de Incertidumbre:** Es un modelo que demuestra cómo la variabilidad de las estimaciones se reduce progresivamente a lo largo del ciclo de vida del proyecto a medida que se adquiere conocimiento técnico y de negocio. | Fase del Proyecto | Factor de Variabilidad / Error | Naturaleza del Conocimiento || ----- | ----- | ----- ||
- Inicio del Proyecto** | High Risk (\$0.25x\$ a \$4.0x\$) | Máxima incertidumbre; alcance genérico y conceptos no probados. || **Definición de Requerimientos** | \$0.5x\$ a \$2.0x\$ | Alcance refinado; se aclaran flujos y criterios de aceptación. || **Diseño Arquitectónico** | \$0.67x\$ a \$1.5x\$ | Arquitectura y componentes definidos; tecnologías seleccionadas. || **Construcción y Despliegue** | \$1.0x\$ (Variabilidad Nula) | Código completado y verificado; el producto está construido. |

Spike Técnico: Cuando una historia de usuario presenta una **incertidumbre o duda técnica desmedidamente alta** (ej. desconocimiento sobre el comportamiento de una API de terceros), no se debe estimar a la ciega ni incluir directamente en el Sprint de desarrollo. En su lugar, se define un **Spike** : una investigación o prototipo acotado en tiempo (*timeboxed*) que realiza el equipo para adquirir el conocimiento técnico necesario, reducir la duda y permitir la posterior estimación e implementación adecuada de la historia.

Pregunta 17: Framework Scrum: Filosofía, Pilares y Roles

Consigna Formal:

Describe el framework Scrum indicando sus pilares empíricos y las responsabilidades específicas de cada uno de sus tres roles constituyentes (Product Owner, Scrum Master y Developers).

Respuesta Desarrollada y Fundamentación Conceptual:

Scrum es un marco de trabajo adaptativo para generar valor en entornos complejos. Se sustenta operativamente en los tres pilares del control de procesos empírico:

Transparencia, Inspección y Adaptación .El *Scrum Team* consta de tres roles específicos sin jerarquías entre sí:

- **Product Owner (PO):**
- Es el responsable directo de **maximizar el valor del producto** resultante del trabajo del equipo.
- Es el único encargado de la gestión efectiva del *Product Backlog* (definir el *Product Goal* , crear y expresar claramente las historias, ordenarlas por prioridad y asegurar su visibilidad).
- **Scrum Master (SM):**
- Es el responsable de **promover y apoyar Scrum** según se define en la Guía Oficial. Actúa como un líder que sirve al equipo y a la organización.

- Facilita los eventos de Scrum, ayuda al equipo a eliminar **impedimentos** (bloqueos operativos) y asegura que el proceso fluya sin interferencias externas.
- **Developers (Desarrolladores):**
- Es el equipo multidisciplinario encargado de crear un **Incremento utilizable y valioso** en cada Sprint.
- Son autoorganizados. Poseen la responsabilidad exclusiva de estimar los ítems del backlog y crear el plan técnico diario (*Sprint Backlog*) para alcanzar el *Sprint Goal* .

Pregunta 18: Framework Scrum: Eventos, Artefactos y Definition of Done (DoD)

Consigna Formal:

Defina los cinco Eventos oficiales de Scrum, sus tres Artefactos con sus respectivos Compromisos según la Guía de Scrum 2020, y analice el concepto de la *Definition of Done* (DoD) como acuerdo formal de calidad.

Respuesta Desarrollada y Fundamentación Conceptual:

Eventos de Scrum (Timeboxes):

1. **Sprint:** El contenedor principal. Iteración de duración fija (1 a 4 semanas) donde se crea un incremento funcional.
2. **Sprint Planning:** Evento de planificación donde se establece qué se puede entregar en el Sprint y cómo se logrará.
3. **Daily Scrum:** Evento diario de 15 minutos para que los Developers inspeccionen el progreso hacia el *Sprint Goal* y adapten su plan.
4. **Sprint Review:** Evento al final del Sprint para inspeccionar el Incremento con los *stakeholders* y adaptar el *Product Backlog* .
5. **Sprint Retrospective:** Evento enfocado en la mejora continua del equipo (procesos, personas, herramientas e interacciones).

Artefactos y sus Compromisos Formales (Guía Scrum 2020):

Cada artefacto contiene un compromiso formal para garantizar la transparencia y la medición del progreso:

- **Product Backlog:** Lista ordenada de todo lo necesario en el producto.
- *Compromiso obligatorio:* **Product Goal (Meta del Producto)** .
- **Sprint Backlog:** Conjunto de ítems seleccionados para el Sprint más el plan para entregarlos.
- *Compromiso obligatorio:* **Sprint Goal (Meta del Sprint)** .
- **Incremento:** La suma de todos los ítems del Product Backlog completados durante el Sprint.
- *Compromiso obligatorio:* **Definition of Done (Definición de Terminado)** .

Definition of Done (DoD):

La **Definition of Done** es una declaración formal que describe el estado de calidad requerido para que un Incremento se considere liberable. Funciona como un acuerdo transparente entre todos los miembros del equipo. Ninguna funcionalidad se considera completada ni se presenta en la *Sprint Review* si no satisface plenamente la DoD (cobertura de pruebas, revisiones de código, documentación técnica e integración).

3. UNIDAD 3: GESTIÓN DE LA CONFIGURACIÓN DEL SOFTWARE (SCM)

Pregunta 19: Introducción a SCM y el Paper de Edward Bersoff

Consigna Formal:

Defina la disciplina de Gestión de la Configuración del Software (SCM - *Software Configuration Management*) y desarrolle los propósitos generales enunciados en el trabajo clásico de Edward Bersoff.

Respuesta Desarrollada y Fundamentación Conceptual:

La **Gestión de la Configuración del Software (SCM)** es la disciplina de la ingeniería de software que aplica una dirección e inspección técnica y administrativa continuas a lo largo del ciclo de vida con el fin de:

1. Identificar unívocamente la estructura y los elementos lógicos y físicos del software.
 2. Controlar de forma sistemática los cambios en dichos elementos.
 3. Registrar e informar el estado de las solicitudes de cambio y su implementación (*Status Reporting*).
 4. Auditar y verificar que los productos resultantes cumplan con los requerimientos establecidos.
- Propósito según Edward Bersoff:** En su trabajo clásico (*Elements of Software Configuration Management*), Edward Bersoff establece que el propósito de SCM es **controlar los cambios para mantener la integridad, trazabilidad y visibilidad del producto de software** a lo largo de su evolución. Dado que el software es intangible y maleable, la ausencia de SCM provoca caos operativo: pérdida de código, sobreescritura de trabajo entre desarrolladores, imposibilidad de reproducir entregas anteriores e incertidumbre sobre qué componentes componen un sistema desplegado en producción.

Pregunta 20: Ítems de Configuración de Software (SCI)

Consigna Formal:

Defina el concepto de Ítem de Configuración de Software (SCI - *Software Configuration Item*) y proporcione ejemplos representativos de los artefactos que deben ser identificados dentro de un proyecto.

Respuesta Desarrollada y Fundamentación Conceptual:

Un **Ítem de Configuración de Software (SCI)** es un elemento de información del software (atómico o compuesto) designado para la gestión de la configuración. Se trata como una entidad individual a los fines de identificación, control de versiones y auditoría dentro del proyecto. **Criterios de Selección:** No todo archivo temporal o borrador es un SCI. Se catalogan como SCI aquellos artefactos críticos cuya modificación no controlada impacte en la calidad, la trazabilidad, la compilación o el despliegue del sistema. **Ejemplos representativos de SCIs en un proyecto:**

- **Artefactos de Código y Ejecución:** Archivos de código fuente (.cs, .java, .py), librerías de terceros, esquemas de bases de datos y *scripts* de migración SQL.
- **Artefactos de Requerimientos y Diseño:** Especificaciones formales, documentos de arquitectura, diagramas de diseño e historias de usuario aprobadas.

- **Artefactos de Verificación y Pruebas:** Planes de prueba, matrices de trazabilidad, casos de prueba y *scripts* de automatización de testing.
- **Artefactos de Operación y Despliegue:** Archivos de configuración por entorno, contenedores (Dockerfile), *scripts* de infraestructura como código (IaC) y manuales de usuario.

Pregunta 21: Líneas Base (Baselines) en SCM

Consigna Formal:

Defina el concepto de Línea Base (*Baseline*) en SCM, analice su utilidad técnico-operativa en la gestión de cambios y aclare las confusiones conceptuales respecto a versiones comerciales y ramas de repositorios (main/trunk).

Respuesta Desarrollada y Fundamentación Conceptual:

Una **Línea Base (Baseline)** es una especificación o producto que ha sido revisado formalmente y acordado, que sirve como base para desarrollo posterior, y que **solo puede ser modificado mediante procedimientos formales de control de cambios** (Tema B, Pregunta 5). **Utilidad Técnico-Operativa:**

- **Capacidad de "Volver Atrás":** Permite reproducir exactamente el entorno y código fuente ejecutable del proyecto en un punto histórico determinado.
- **Estabilidad de Desarrollo:** Proporciona un punto de partida congelado y confiable para que los desarrolladores construyan nuevos componentes sin sufrir perturbaciones por cambios no autorizados.

Aclaraciones y Distinciones Evaluadas en Exámenes Teóricos:

1. **Diferencia con la Versión Comercial:** Una *Baseline* no debe confundirse con la versión comercial del producto vendida al público (ej. Versión 3.1). Múltiples *baselines* técnicas intermedias se generan internamente antes de producir un lanzamiento comercial.
2. **Diferencia con la Rama Principal (main / trunk):** Una *Baseline* **no es equivalente a la rama principal de Git** . La rama main es una estructura de código técnica que puede cambiar constantemente. Una *Baseline* es un estado formalmente aprobado que congela no solo código, sino también la documentación, pruebas y configuraciones mediante un procedimiento administrativo.

Pregunta 22: Estructura del Repositorio de SCM: Ruta de Producto vs. Ruta de Proyecto

Consigna Formal:

Explique la organización arquitectónica del repositorio SCM diferenciando las responsabilidades de almacenamiento entre la "Ruta de Producto" (*Product Path*) y la "Ruta de Proyecto" (*Project Path*).

Respuesta Desarrollada y Fundamentación Conceptual:

Para evitar la contaminación de entregables y garantizar la trazabilidad, la estructura organizativa del repositorio de SCM debe dividirse en dos rutas principales:

- **Ruta de Producto (*Product Path*):**
- **Propósito:** Contiene exclusivamente los Ítems de Configuración de Software (SCIs) que **forman parte directa de la solución ejecutable entregable** o que son indispensables para compilarla, construirla, desplegarla y operarla.

- **Artefactos incluidos:** Código fuente, componentes compilados, librerías de terceros, esquemas de bases de datos, *scripts* de migración, *Dockerfiles*, configuraciones de entorno e instructivos de despliegue.
- **Ruta de Proyecto (*Project Path*):**
- **Propósito:** Almacena todos los elementos de gestión y documentación administrativa generados para dirigir el proceso del proyecto, los cuales **no se compilan ni se empaquetan en el software final entregado al cliente**.
- **Artefactos incluidos:** Planes de proyecto, presupuestos, cronogramas, minutas de reunión, actas de conformidad, matrices de riesgo e informes de métricas de desempeño.

Pregunta 23: Flujo de Control de Cambios y el Comité CCB

Consigna Formal:

Describe las fases del Flujo de Control de Cambios en SCM y defina la composición y responsabilidades del Comité de Control de Cambios (CCB - *Change Control Board*).

Respuesta Desarrollada y Fundamentación Conceptual:

El **Flujo de Control de Cambios** es el procedimiento formal para evaluar e incorporar modificaciones en una *baseline* aprobada sin comprometer la estabilidad del sistema:

1. **Solicitud de Cambio (CR - *Change Request*):** Registro formal de la propuesta de cambio emitida por el cliente, usuario o desarrollador, justificando la necesidad.
 2. **Evaluación de Impacto:** Análisis técnico y operativo sobre el costo, tiempo, riesgos, arquitectura y esfuerzo requerido.
 3. **Decisión / Aprobación:** Instancia donde la autoridad competente aprueba, rechaza o difiere la solicitud.
 4. **Implementación:** Asignación del trabajo técnico para modificar los SCIs correspondientes en el entorno de desarrollo.
 5. **Verificación y Auditoría:** Comprobación de que el cambio fue implementado correctamente sin generar regresiones en el producto.
- El Comité de Control de Cambios (CCB - *Change Control Board*):** El CCB es la **autoridad formal asignada** para evaluar la conveniencia e impacto de cada cambio propuesto sobre las líneas base.
- **Composición Multidisciplinaria:** Está constituido por el Líder de Proyecto, Arquitecto de Software, Responsable de Calidad (SQA), Gestor de Configuración (SCM), Representante del Cliente / Product Owner y Líder de Pruebas.

Pregunta 24: Auditorías de Configuración: FCA y PCA

Consigna Formal:

Diferencie teórica y prácticamente las dos formas principales de auditorías en la Gestión de la Configuración del Software: la Auditoría de Configuración Funcional (FCA) y la Auditoría de Configuración Física (PCA).

Respuesta Desarrollada y Fundamentación Conceptual:

Las auditorías de configuración son revisiones formales que aseguran que el producto construido coincida plenamente con su documentación técnica. Se clasifican en dos auditorías complementarias:

- **FCA (Functional Configuration Audit - Auditoría de Configuración Funcional):**
- **Pregunta Clave:** ¿El producto cumple funcionalmente con lo especificado?
- **Mecanismo:** Verifica que el ítem de configuración haya completado satisfactoriamente todas las pruebas funcionales. Comprueba los informes de ejecución de pruebas y matrices de trazabilidad para garantizar que cada requerimiento especificado funcione correctamente.
- **PCA (Physical Configuration Audit - Auditoría de Configuración Física):**
- **Pregunta Clave:** ¿La estructura física del producto "as-built" coincide con la documentación de diseño?
- **Mecanismo:** Inspecciona físicamente los componentes del software en el repositorio, verificando que todos los SCIs (código, manuales, archivos de configuración, instaladores) estén correctamente catalogados, etiquetados y versionados sin omitir elementos ni incluir archivos no autorizados.

Pregunta 25: Informes de Estado (Status Reporting) y SCM en Ambientes Continuos (CI/CD)

Consigna Formal:

Explique el rol de los Informes de Estado de la Configuración (*Status Reporting*) y analice cómo se integra la disciplina de SCM con las prácticas modernas de Integración Continua, Entrega Continua y Despliegue Continuo (CI/CD).

Respuesta Desarrollada y Fundamentación Conceptual:

Informes de Estado de la Configuración (*Status Reporting* / *Status Accounting*): La contabilidad del estado de la configuración es la función de SCM encargada de registrar e informar la evolución de los artefactos. Mantiene el registro histórico de:

- El estado exacto de cada Solicitud de Cambio (CR) (propuesta, evaluada, aprobada, rechazada o implementada).
 - Las versiones aprobadas de los SCIs que componen cada *baseline* .
 - La trazabilidad histórica entre requerimientos, cambios y entregas producidas.
- Integración de SCM en Ambientes Continuos (CI/CD):** En las prácticas modernas de desarrollo de software, la SCM se integra con la automatización de las tuberías (*pipelines*) de **CI/CD** :
- **Integración Continua (CI):** Ante cada confirmación de cambios (*commit*) realizada por un desarrollador en el repositorio central de SCM, la tubería ejecuta automáticamente procesos de compilación y pruebas unitarias. SCM aporta la trazabilidad inmediata del autor y motivo de cada cambio.
 - **Entrega y Despliegue Continuo (CD):** Los artefactos validados se empaquetan y despliegan de forma automatizada hacia entornos de prueba o producción.
 - **Gobernanza Automatizada:** La automatización de CI/CD no elimina la SCM, sino que la potencia: garantiza que el control de versiones, la trazabilidad y los controles de calidad se ejecuten en segundos de forma continua, eliminando la burocracia manual sin descuidar la integridad del sistema.